ORCH: A Zero-API-Key, Multi-Surface, Dual-Category Agentic Coding Harness

Technical Whitepaper & Complete Architectural Reference (v2.0)

Author: Aswath (`aswathas` / `aswathas20@gmail.com`)

Repository: `agy-orchestrator`

Date: November 10, 2026

Abstract

Modern Large Language Models (LLMs) and terminal coding agents—such as Google Antigravity (`agy`), Command Code (`cmdc`), and MiniMax Code (`mcode`)—are individually capable code generators, yet they fail systematically on complex, long-horizon engineering work when operated as isolated CLI sessions. Single-model sessions suffer from confirmation bias, context window pollution ("Lost in the Middle"), cross-OS boundary failures between WSL2 Linux and Windows NT, stateless amnesia across tools, and static hierarchy lock-in.

This whitepaper presents **ORCH (`agy-orchestrator`)**, a lightweight, zero-API-key command orchestrator harness that unifies **AGY WSL**, **AGY Windows**, **Command Code (`cmdc`)**, and **MiniMax Code (`mcode`)** into a dynamically reconfigurable multi-agent system. ORCH coordinates models across two distinct foundation families—**Category A (Google Gemini: `gemini-4-argon-*`, `gemini-3.8-flash-*`)** and **Category B (Anthropic Claude / OpenAI GPT: `claude-opus-5-5-*`, `claude-sonnet-5-5-*`, `gpt-oss-120b-medium`)**—without requiring paid external API keys.

We detail every subsystem from first principles down to individual functions and equations:

- **What an Agentic Harness Is** (and why models without a harness fail).
- **Cross-OS Process & Filesystem Bridging** across WSL2 Linux (`ext4`) and Windows (`NTFS` over `9p`).
- **Live-Swappable Leader/Subagent Topology** with automatic **Elephant-Goldfish Model (EGM)** adversarial pairing.
- **3-Tier MemGPT + `claude-mem` Hybrid Memory** using SQLite FTS5 BM25, 384-dimensional `numpy` dense vector embeddings, **Reciprocal Rank Fusion (RRF)**, and Ebbinghaus temporal decay across **9,541+ historical observations**.
- **Two-Stage Thinking Skill Router & Thompson Sampling Multi-Armed Bandit** indexing **188 local skills** in `<10ms` with on-demand skill acquisition (`npx skills`).
- **User Taste Engine & Contrastive DPO Learning Loop** enforcing `ponytail:ultra` minimalism and continuous rule consolidation.
- **Bi-Directional Phone & Cloud Notes Automation** via atomic Markdown state machines (`OneDrive/Orch-Phone-Inbox.md`) and `ntfy.sh` pub/sub streams.
- **Zero-Build Tactical Telemetry Web Control Center & Self-Improviser Studio** (`http://127.0.0.1:4888`).

Table of Contents

- [Part I: First Principles — What Is an Agentic Harness?](#part-i-first-principles--what-is-an-agentic-harness)
- [Part II: High-Level Architecture & Zero-API-Key Economics](#part-ii-high-level-architecture--zero-api-key-economics)
- [Part III: Cross-OS Execution Bridge (WSL2 Linux ↔ Windows NT)](#part-iii-cross-os-execution-bridge-wsl2-linux--windows-nt)
- [Part IV: Dynamic Topology Controller & The Elephant-Goldfish Protocol](#part-iv-dynamic-topology-controller--the-elephant-goldfish-protocol)
- [Part V: 3-Tier MemGPT + `claude-mem` Hybrid Memory Engine](#part-v-3-tier-memgpt--claude-mem-hybrid-memory-engine)
- [Part VI: Two-Stage Thinking Skill Router & Contextual Thompson Bandit](#part-vi-two-stage-thinking-skill-router--contextual-thompson-bandit)

- [Part VII: User Taste Engine & Contrastive DPO Learning Loop](#part-vii-user-taste-engine--contrastive-dpo-learning-loop)
- [Part VIII: Phone & Cloud Notes Bi-Directional Automation Agent](#part-viii-phone--cloud-notes-bi-directional-automation-agent)
- [Part IX: Tactical Telemetry Web Control Center & Self-Improviser](#part-ix-tactical-telemetry-web-control-center--self-improviser)
- [Part X: Atomic Component Breakdown, File Map & CLI Reference](#part-x-atomic-component-breakdown-file-map--cli-reference)
- [Part XI: Engineering Chronicle — Decisions, Failures & Pivots](#part-xi-engineering-chronicle--decisions-failures--pivots)

Part I: First Principles — What Is an Agentic Harness?

1.1 Model vs. Agent vs. Harness

To understand why **ORCH** exists, we must separate three layers that are frequently conflated in AI engineering:

Layer	Definition	What It Can Do	What It Cannot Do
1. Foundation Model (LLM)	A stateless token predictor $P(y_t \mid y_{<t}, x)$ (e.g., `gemini-4-argon-high`, `claude-opus-5-5-high`, `gpt-oss-120b-medium`).	Reason over text in its current context window and emit text or structured tool calls.	Execute commands, read files, remember past sessions, or verify its own claims against reality.
2. Single CLI Agent	A binary wrapper around a model (`agy`, `cmdc`, `mcode`, `claude`) with a tool execution loop (`Read`, `Edit`, `Bash`).	Complete bounded single-session tasks inside one OS environment using one model family.	Coordinate across OS boundaries (WSL + Windows), critique itself with an independent model family, share unified ranked memory across different CLI tools, or dynamically route across 188 skills without blowing up context.
3. Agentic Harness (`ORCH`)	The **operating system and control plane above CLI agents**. It governs memory, skill routing, topology, taste enforcement, cross-family verification, and learning.	Orchestrate multiple CLI agents across WSL and Windows, combine Gemini + Claude/GPT strengths, retrieve ranked hybrid memory in milliseconds, learn from mistakes via reinforcement bandits, and accept remote tasks from a phone.	N/A — the harness is the meta-system that turns isolated CLI binaries into a cohesive, self-improving engineering organization.

1.2 The Six Failure Modes of Un-Harnessed Agents

When a developer runs a single CLI agent without a harness, six predictable failure modes emerge:

- **Single-Family Confirmation Bias (The Echo Chamber)**:

  When the same model drafts a plan and reviews its own code in the same conversation context, it shares the same blind spots and implicit assumptions. It validates broken designs because its prior tokens condition its review tokens.

- **Naive Context Stuffing ("Lost in the Middle")**:

  Dumping a flat `global-memory.json` or 85+ `SKILL.md` files into an agent's system prompt wastes 20,000+ tokens per turn and degrades reasoning accuracy on the middle 60% of the prompt.

- **Cross-OS Schism (WSL2 vs. Windows NT)**:

  On a developer workstation running Windows 11 + WSL2 Ubuntu, some tools and workspaces live in Linux (`/home/aswath/...`) while others live in Windows (`C:\Users\aswat\...`, OneDrive, Windows-native `agy.exe`, `cmdc.cmd`). Un-harnessed agents fail on UNC paths (`\\wsl.localhost\...`) and corrupt SQLite WAL locks over the `/mnt/c 9p` filesystem.

- **Cross-Tool Amnesia**:

  Observations learned during a Claude Code session (`claude-mem.db`) are invisible to `agy` (WSL/Windows) and `cmdc`, forcing the human to repeat corrections across tools.

- **Taste Drift & AI Slop**:

  Left unchecked, LLMs revert to statistical averages: verbose boilerplate, `// TODO` placeholders, generic purple-gradient UIs (`Inter` font, 3-column cards), and overwritten `git config user.name` authorship.

- **Static Orchestration Lock-In**:

  Most multi-agent frameworks hardcode which model is the "Planner" and which is the "Worker." When quotas shift, task types change, or a new model drops, the human cannot swap the Leader and Subagents on the fly.

ORCH solves all six failure modes in ~1,800 lines of zero-dependency Node.js (ESM) and Python (`sqlite3` + `numpy`).

Part II: High-Level Architecture & Zero-API-Key Economics

2.1 Zero-API-Key Execution Principle

Paying per-token API costs for a multi-agent swarm (Router + Leader + 2 Subagents + Critic) quickly becomes prohibitively expensive for an individual builder. However, modern developer CLIs (`agy` on WSL, `agy.exe` on Windows, `cmdc`, and `mcode`) already provide authenticated, subscription-backed headless execution flags:

- **AGY WSL (`/home/aswath/.local/bin/agy`)**:

```
agy -p "<prompt>" --model <gemini|claude|gpt> --effort <low|medium|high|xhigh|max> --output-format text --dangerously-skip-permissions
```

- **AGY Windows (`C:\Users\aswat\AppData\Local\agy\bin\agy.exe`)**:

  Same flags, executed natively in the Windows NT environment.

- **Command Code (`cmdc`)**:

```
cmdc -p "<prompt>" --yolo --skip-onboarding --skill <skill-dir> --output-format text
```

- **MiniMax Code (`mcode`)**:

```
mcode exec --append-system-prompt-file <context.md> "<prompt>"
```

By treating each installed CLI binary as a **Pluggable Brain Adapter**, ORCH achieves full multi-model, multi-OS orchestration at **$0.00 in external API key spend**.

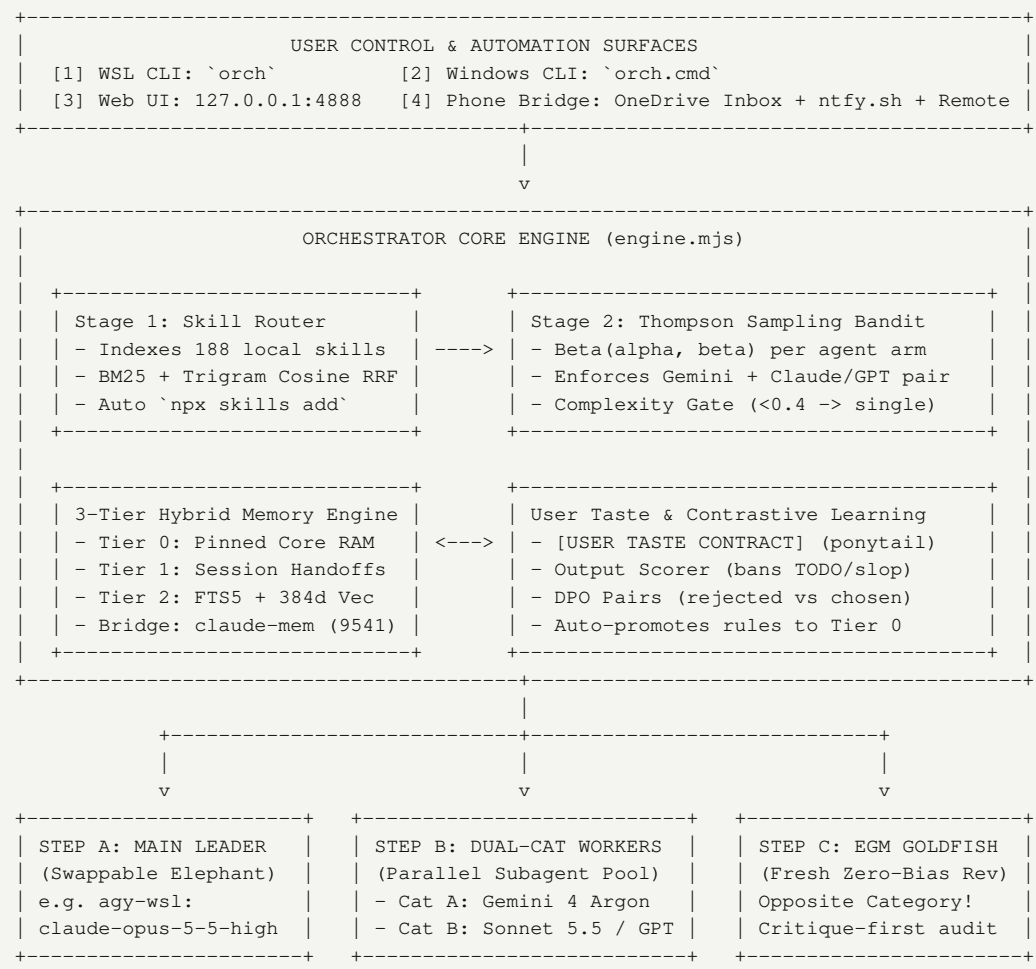
2.2 Dual-Category Model Pool Inside AGY

Within `agy` (both WSL and Windows), models fall into two distinct architectural families that ORCH actively pairs against each other:

Category	Model ID	Human Name	Primary Strength in ORCH
Category A: Gemini	`gemini-4-argon-high`	Gemini 4 Argon (High)	Deep architecture, long-context synthesis, EGM Goldfish review, UI/UX design
Category A: Gemini	`gemini-4-argon-medium`	Gemini 4 Argon (Medium)	Balanced full-stack implementation & cross-file reasoning
Category A: Gemini	`gemini-4-argon-low`	Gemini 4 Argon (Low)	Rapid iteration and low-latency sub-tasks
Category A: Gemini	`gemini-3.8-flash-high`	Gemini 3.8 Flash (High)	High-speed Thinking Router, verification, and parallel worker fan-out
Category A: Gemini	`gemini-3.8-flash-medium`	Gemini 3.8 Flash (Medium)	Windows worker tasks, quick validation, test runners
Category A: Gemini	`gemini-3.8-flash-low`	Gemini 3.8 Flash (Low)	Sub-second triage and lightweight formatting
Category B: Claude/GPT	`claude-opus-5-5-high`	Claude Opus 5.5 (High)	Main Elephant Leader, complex systems design, deep debugging
Category B: Claude/GPT	`claude-opus-5-5-medium`	Claude Opus 5.5 (Medium)	High-precision code generation and specification drafting
Category B: Claude/GPT	`claude-opus-5-5-low`	Claude Opus 5.5 (Low)	Fast Opus pass
Category B: Claude/GPT	`claude-sonnet-5-5-high`	Claude Sonnet 5.5 (High)	Primary implementation coder, refactoring, TDD loops
Category B: Claude/GPT	`claude-sonnet-5-5-medium`	Claude Sonnet 5.5 (Medium)	General coding and script authoring

Category B: Claude/ GPT	`claude-sonnet-5-5-low`	Claude Sonnet 5.5 (Low)	Quick edits
Category B: Claude/ GPT	`gpt-oss-120b-medium`	GPT-OSS 120B (Medium)	Alternative implementation perspective, independent code review

2.3 End-to-End System Topology Diagram



Part III: Cross-OS Execution Bridge (WSL2 Linux ↔ Windows NT)

3.1 The Two Cross-OS Bugs That Break Naive Harnesses

When orchestrating between WSL2 Ubuntu and Windows 11, two low-level OS bugs break standard subprocess and database code:

- **Bug #1 — `cmd.exe` UNC Current Working Directory Rejection**:

  When a Node.js process running inside WSL (`/home/aswath/...`) spawns `cmd.exe /c agy.exe -p "..."`, Windows sees the current working directory as `\\wsl.localhost\Ubuntu\home\aswath`. `cmd.exe` immediately prints:

```
'\\wsl.localhost\Ubuntu\home\aswath'
CMD.EXE was started with the above path as the current directory.
UNC paths are not supported. Defaulting to Windows directory.
```

Worse, complex multi-line prompts containing quotes, newlines, or markdown backticks get mangled by `cmd.exe` metacharacter expansion.

- **Bug #2 — WSL2 `9p` (`drvfs`) SQLite WAL Shared-Memory Lock Failure**:

  Your Windows `claude-mem` database lives at `C:\Users\aswat\.claude-mem\claude-mem.db` (mounted in WSL at `/mnt/c/Users/aswat/.claude-mem/claude-mem.db`) in SQLite `WAL` mode (`claude-mem.db-wal` and `claude-mem.db-shm`). When Python inside WSL runs `sqlite3.connect('/mnt/c/Users/aswat/.claude-mem/claude-`

`mem.db')`, the Linux kernel attempts an `fcntl` byte-range lock on the `-shm` file across the `9p` network filesystem protocol, which immediately throws:

```
sqlite3.OperationalError: disk I/O error
```

3.2 How ORCH Solves Both (`src/adapters/path-bridge.mjs` & `src/adapters/index.mjs`)

- **The Staging Bridge & PowerShell Raw Reader (`ensureBridgeDir` + `buildBrainCommand`)**:
- ORCH creates a shared staging directory at `/mnt/c/Users/aswat/.agy-orch-bridge` (`C:\Users\aswat\.agy-orch-bridge`).
- Before invoking `agy-win`, ORCH writes the full assembled prompt (Taste Contract + RRF Memory + Mounted Skills + Task) as UTF-8 to `C:\Users\aswat\.agy-orch-bridge\prompt-<id>.txt`.
- It spawns `cmd.exe` with `cwd: '/mnt/c/Users/aswat'` (`C:\Users\aswat`, a native NTFS path, eliminating the UNC error) and invokes PowerShell to read the file verbatim into memory without shell parsing:

```
$p = Get-Content -Raw -Encoding UTF8 'C:\Users\aswat\.agy-orch-bridge\prompt-123.txt';
& 'C:\Users\aswat\AppData\Local\agy\bin\agy.exe' -p $p --model "gemini-4-argon-high" --effort high --
output-format text --dangerously-skip-permissions
```

- **Split-Storage SQLite Topology (`orch-mem.db` on `ext4` + `?immutable=1` on `claude-mem.db`)**:
- ORCH's primary write database (`orch-mem.db`) lives on native Linux `ext4` at `/home/aswath/agy-orchestrator/memory/orch-mem.db`, where `PRAGMA journal_mode=WAL` runs with native POSIX kernel locks at full speed.
- Whenever ORCH queries your Windows `claude-mem.db` (9,541 observations), it opens a short-lived URI connection using `file:/mnt/c/Users/aswat/.claude-mem/claude-mem.db?immutable=1`, bypassing `-shm` lock acquisition completely while reading the full FTS5 index (`observations_fts`) in `<5ms`.

Part IV: Dynamic Topology Controller & The Elephant-Goldfish Protocol

4.1 Non-Static Hierarchy (`src/topology/controller.mjs`)

Unlike static frameworks where roles are hardcoded in Python or TypeScript classes, ORCH stores its live hierarchy in [`config/topology.json`](../../config/topology.json) managed by `TopologyController`:

- **`leader`**: The main architect and coordinator (e.g., `agy-wsl:claude-opus-5-5-high` or `agy-win:gemini-4-argon-high` or `cmdc:default`).
- **`goldfishCritic`**: The fresh-context reviewer.
- **`router`**: The fast routing brain (`agy-wsl:gemini-3.8-flash-high`).
- **`tasteCritic`**: The taste gate (`cmdc:default`).
- **`subagents`**: A named map of workers (`gemini-architect`, `win-gemini-worker`, `claude-coder`, `gpt-worker`, `cmdc-worker`, `mcode-fast`), each tagged with `surface`, `category`, `model`, `effort`, and `specialties`.

4.2 Automatic Opposite-Category Goldfish Alignment

In the **Elephant-Goldfish Model (EGM)**:

- **The Elephant (`leader`)** holds your full project context, hybrid memories, and skills to draft the plan or code.
- **The Goldfish (`goldfishCritic`)** inspects *only* the problem statement and the Elephant/Workers' draft in a **fresh context with zero conversation history**, exposing unstated assumptions and edge-case bugs.

ORCH goes one step further than standard EGM: **when you change the Main Leader (`orch set-leader` or `orch swap`), `TopologyController` automatically switches `goldfishCritic` to the opposite model category**:

- If `leader.category === "claude-gpt"` (e.g. `claude-opus-5-5-high`), `goldfishCritic` automatically becomes **`gemini-4-argon-high`** (`category: "gemini"`).
- If `leader.category === "gemini"` (e.g. `gemini-4-argon-high`), `goldfishCritic` automatically becomes **`claude-opus-5-5-high`** (`category: "claude-gpt"`).

This guarantees that the drafter and the reviewer never belong to the same model family.

Part V: 3-Tier MemGPT + `claude-mem` Hybrid Memory Engine

5.1 Why Simple JSON Memory Fails

A flat JSON file (`global-memory.json`) has three fatal flaws at scale:

- **Linear Token Growth ($O(N)$)**: Every added note increases prompt size until context limits or attention degradation hit.
- **Zero Lexical + Semantic Precision**: `Array.filter()` cannot rank exact code symbols alongside conceptual paraphrases.
- **No Temporal or Salience Weighting**: A stale debugging note from 3 weeks ago receives the same weight as an architectural invariant pinned 5 minutes ago.

5.2 The 3-Tier MemGPT Architecture (`src/memory/hybrid_engine.py`)

ORCH implements a 3-tier memory hierarchy inspired by MemGPT and cognitive memory models:

- **Tier 0 — Core Pinned RAM (`taste-profile.json` + Pinned Invariants)**:

  Always resident in the system prompt (~450 tokens). Holds your compiled `[USER TASTE CONTRACT]` and promoted hard rules.

- **Tier 1 — Session Working Memory & Handoff Ledger (`handoffs` table in `orch-mem.db`)**:

  Stores structured session handoffs (`state`, `next_step`, `files_json`, `open_questions`, `cli_conversations_json`) modeled after your `HarnessAgents/hive/PROTOCOL.md`.

- **Tier 2 — Archival Hybrid Store (`memories` + `memories_fts` + `claude-mem.db` bridge)**:

  Combines local cross-OS memories with your existing **9,541 observations** in `claude-mem.db`.

5.3 Feature-Hashed N-Gram Dense Vector Embeddings (Zero API Cost, `<2ms`)

To compute dense vector similarity without calling paid OpenAI/Cohere embedding APIs or waiting for heavy GPU model loads, `hybrid_engine.py` implements **Signed Feature-Hashed Word + Character N-Gram Embeddings** in `numpy` (`embed_text`):

- Given text T, extract normalized word tokens w_i, word bigrams $b_i = (w_i, w_{i+1})$, and boundary-padded character 3-grams and 4-grams c_j from `^token$`.
- Assign base weights: word unigrams $= 1.5$, word bigrams $= 1.2$, character 3/4-grams $= 0.5$ (capturing morphological stems, typos, and camelCase/snake_case code identifiers).
- Apply sublinear term-frequency scaling:

  $$\text{tf_weight}(f) = 1 + \ln(\text{count}(f))$$

- Project into $\mathbb{R}^{384}$ via signed CRC32 feature hashing (Weinberger et al., *Feature Hashing for Large Scale Multitask Learning*):

  $$h = \text{CRC32}(f), \quad \text{bucket} = h \bmod 384, \quad \text{sign} = (-1)^{(h \gg 16) \& 1}$$

  $$v[\text{bucket}] \mathrel{+}= \text{sign} \cdot \text{tf_weight}(f)$$

- Normalize to the unit hypersphere:

  $$\hat{v} = \frac{v}{\|v\|_2 + \epsilon} \in \mathbb{R}^{384}$$

Stored as a 1,536-byte `float32` BLOB in SQLite, cosine similarity across hundreds of memories is a single vectorized `np.dot(V, q_vec)` taking `<1ms`.

5.4 Reciprocal Rank Fusion (RRF) + Ebbinghaus Forgetting Curve

When searching memory for a task query q, ORCH computes two rankings:

- **Lexical Rank $\text{rank}_{\text{BM25}}(d, q)$** from SQLite's native **FTS5** (`bm25(memories_fts)` with Porter stemming).
- **Dense Vector Rank $\text{rank}_{\text{cos}}(d, q)$** ordered by cosine similarity $\cos(\hat{v}_d, \hat{v}_q)$.

It then fuses both rankings with **Reciprocal Rank Fusion (RRF)** modulated by **Ebbinghaus Exponential Time Decay**, **Logarithmic Access Reinforcement**, and **Salience s_d**:

$$\text{Score}(d, q) = \left(\frac{0.45}{60 + \text{rank}_{\text{BM25}}(d, q)} + \frac{0.55 \cdot \max(0.1, \cos(\hat{v}_d,

\hat{v}_q))}{60 + \text{rank}_{\text{cos}}(d, q)} \right) \cdot e^{-\lambda \Delta t_d} \cdot \left(1 + 0.15 \ln(1 + \text{hits}_d)\right) \cdot s_d$$

Where:

- Δt_d is the age in hours since memory d was updated ($\lambda = 0.002$).
- hits_d is incremented every time memory d is retrieved into a top-K context window (frequently useful memories resist decay).
- Simultaneously, `query_claude_mem(q)` queries `observations_fts` in `/mnt/c/Users/aswat/.claude-mem/claude-mem.db?immutable=1` and merges the top matches into the unified RRF result list.

Part VI: Two-Stage Thinking Skill Router & Contextual Thompson Bandit

6.1 Stage 1: Sub-10ms Local Skill Indexing & Hybrid RRF Selection (`src/router/skill-router.mjs`)

Your machine has **188 skills** across:

- `/home/aswath/.agents/skills`
- `/home/aswath/.claude/skills`
- `/mnt/c/Users/aswat/.claude/plugins/marketplaces/egm/skills`

`SkillRouter.indexLocalSkills()` parses the YAML frontmatter (`name`, `description`) of all 188 `SKILL.md` files, builds an in-memory inverted document-frequency (`df`) table and character-trigram map, and ranks skills for any prompt q using:

$$\text{SkillScore}(s, q) = \left(\frac{0.55}{20 + \text{rank}_{\text{BM25}}(s, q)} + \frac{0.45 \cdot \text{sim}_{\text{3gram}}(s, q)}{20 + \text{rank}_{\text{3gram}}(s, q)} \right) \cdot \left(0.85 + 0.30 \cdot \frac{\alpha_s}{\alpha_s + \beta_s}\right)$$

Where $\frac{\alpha_s}{\alpha_s + \beta_s}$ is the learned historical success rate of skill s from the Learning Loop!

- **On-Demand Skill Acquisition Gate**: If the top skill score falls below $\tau = 0.008$ (or `--install` is requested), `SkillRouter.installMissingSkill(pkg)` invokes `npx --yes skills add <pkg> -g -y` or `synthesizeSkill()` to create a new `SKILL.md` in `~/.agents/skills/` and re-indexes immediately.

6.2 Complexity Gate (Preventing Over-Orchestration)

`extractTaskFeatures(prompt)` extracts an 8-dimensional binary intent vector:

$$\vec{f}(q) \in \{0, 1\}^8 = [\text{architecture}, \text{implementation}, \text{debugging}, \text{frontend_ui}, \text{windows_os}, \text{wsl_linux}, \text{research}, \text{refactor}]$$

And computes a normalized **Complexity Score**:

$$C(q) = \min\left(1.0,\; 0.16 \sum_{i=1}^{8} f_i(q) + \min\left(0.40,\; \frac{\text{words}(q)}{60}\right)\right)$$

- If $C(q) < 0.40$ (e.g., a quick question or 1-line edit), the Complexity Gate routes to **`single` (Fast-Track)** mode so you don't wait for 4 subagents.
- If $C(q) \ge 0.40$, it triggers **`full` Dual-Category** or **`egm` Elephant+Goldfish** fan-out.

6.3 Stage 2: Contextual Thompson Sampling Multi-Armed Bandit (`src/learning/loop.mjs`)

To decide *which* subagents from the pool should execute a task, ORCH models each subagent i as an arm in a **Contextual Multi-Armed Bandit** with a Beta posterior $\text{Beta}(\alpha_i, \beta_i)$:

- **Posterior Sampling (Marsaglia & Tsang Exact Gamma Sampler)**:

  For each candidate worker i, sample:

  $$X \sim \text{Gamma}(\alpha_i, 1), \quad Y \sim \text{Gamma}(\beta_i, 1) \implies \theta_i = \frac{X}{X + Y} \sim \text{Beta}(\alpha_i, \beta_i)$$

- **Contextual Specialty Alignment**:

  Compute dot-product overlap $\text{match}_i(q)$ between the worker's `specialties` and the task's feature vector $\vec{f}(q)$, yielding total selection priority:

  $$S_i(q) = 0.60 \cdot \theta_i + 0.40 \cdot \text{match}_i(q)$$

- **Dual-Category Constraint Enforcement**:

After picking the highest-scoring worker, `selectAgentsBandit()` checks the set of participating model categories (`leader.category` $\cup$ `selectedWorkers.category`). If either **`gemini`** or **`claude-gpt`** is missing, it automatically pulls in the highest-scoring worker from the missing category—ensuring every complex task benefits from both Gemini and Claude/GPT intelligence.

Part VII: User Taste Engine & Contrastive DPO Learning Loop

7.1 The Compiled `[USER TASTE CONTRACT]` (`src/taste/engine.mjs`)

Every prompt dispatched by ORCH prepends a deterministic contract compiled from [`taste/taste-profile.json`](../../taste/taste-profile.json):

- **Coding Discipline**: `ponytail:ultra` (smallest complete change, standard library first, zero dead abstractions).
- **Anti-Truncation Guard**: Bans `// TODO`, `// TBD`, and `... [truncated]` placeholders.
- **UI Engineering Baseline**: Enforces `industrial-brutalist-ui` Archetype 2.2 (`#0A0A0A` carbon substrate, `#EAEAEA` phosphor foreground, `#E61919` aviation red accent, `0px` border-radius, `1px` CSS grid, `Variance=8`, `Motion=6`, `Density=8`).
- **Git Authorship Preservation**: Injects `GIT_AUTHOR_NAME` and `GIT_COMMITTER_NAME` so CLI agents never overwrite your GitHub commit credit.

7.2 Online Bandit Update & Contrastive DPO Consolidation (`src/learning/loop.mjs`)

After every agent execution:

- **Reward Calculation**:

  $$r = \begin{cases} 0.60 \cdot 1.0 + 0.25 \cdot \text{TasteScore}(y) + 0.15 \cdot \max\left(0, 1 - \frac{\text{latencyMs}}{60000}\right) & \text{if exitCode} = 0 \\ 0.05 & \text{otherwise} \end{cases}$$

- **Posterior Update**:

  $$\alpha_i \leftarrow \alpha_i + r, \qquad \beta_i \leftarrow \beta_i + (1 - r)$$

- **Contrastive Preference Pairs (DPO-style)**:

  Whenever an approach fails or you record a correction (`orch learn`), `recordContrastivePair()` logs `{ cluster, rejected, chosen, rootCause, utility }` to `learning/records.jsonl`.

- **Rule Consolidation (`consolidateRules`)**:

  Any contrastive pair with $\text{utility} \ge 0.80$ is automatically distilled into a permanent rule inside `taste/taste-profile.json` (`customRules`), immediately governing all future agent runs across WSL and Windows.

Part VIII: Phone & Cloud Notes Bi-Directional Automation Agent

8.1 Three Phone-to-Harness Channels (`src/automation/phone-bridge.mjs`)

You can trigger and monitor ORCH from your phone anywhere without running a public cloud server:

- **Channel A — Cloud-Synced Notes Inbox (`/mnt/c/Users/aswat/OneDrive/Orch-Phone-Inbox.md`)**:
- Synced automatically with your phone via OneDrive (or Google Keep / Drive / Obsidian export).
- **Atomic 3-State Markdown Machine**:
- You add `- [ ] Build a benchmark script` on your phone.
- `PhoneAutomationBridge.processInboxFile()` detects `- [ ]`, **atomically rewrites it on disk to `- [~]` before spawning any agent** (preventing duplicate runs on file-watcher ticks), executes the full ORCH pipeline, and rewrites the line to:

```
- [x] Build a benchmark script
  > **Orch Result (2026-11-10T08:00:00Z · full · 3 step(s))**: ...
```

- You open the same note on your phone and read the result inline.
- **Channel B — Zero-Account `ntfy.sh` Pub/Sub Stream**:
- Polls `https://ntfy.sh/agy-orch-aswath-cmd-9842/json` for incoming phone commands and POSTs

completion notifications to `https://ntfy.sh/agy-orch-aswath-out-9842`.

- Works with the free open-source `ntfy` Android/iOS app or any mobile HTTP shortcut.
- **Channel C — Native `agy remote-control` Integration**:
- Directly starts, checks, or stops the built-in `agy remote-control` daemon from the CLI or Web UI.

Part IX: Tactical Telemetry Web Control Center & Self-Improviser

9.1 Zero-Build HTTP + SSE Architecture (`src/ui/server.mjs` & `src/ui/index.html`)

Running `orch ui` launches a zero-dependency Node HTTP server on **`http://127.0.0.1:4888`** with a live **Server-Sent Events (`/api/events`)** telemetry bus.

Built strictly to your `industrial-brutalist-ui` (Archetype 2.2: Tactical Telemetry & CRT Terminal) and `design-taste-frontend` specifications:

- **Palette**: `#0A0A0A` deactivated CRT carbon background, `#121212` cell background, `#EAEAEA` white phosphor text, `#E61919` aviation hazard red structural accent, and a single `#4AF626` terminal green heartbeat dot.
- **Typography**: `Archivo Black` uppercase macro headers (`letter-spacing: -0.04em`) + `JetBrains Mono` (`12px`) tabular telemetry.
- **Six Interactive Workspaces**:
- `01 // TOPOLOGY & LEADER`: Switch Leader surface/model/effort, one-click **Promote** / **Swap Leader** on any subagent card, add/remove workers, sync AGY CLI themes (`tokyo night`, `solarized dark`, etc.) across WSL & Windows.
- `02 // TASK DISPATCH & EGM`: Run tasks in `adaptive`, `full`, `egm` (Elephant+Goldfish), or `single` mode with live SSE streaming.
- `03 // CLAUDE-MEM & RRF`: Live search bar across your **9,541 `claude-mem` observations** + `orch-mem.db`, showing exact `RRF`, `BM25_RANK`, and `COSINE` scores.
- `04 // SKILL ROUTER & BANDIT`: Simulate routing on any prompt, install skills from `npx skills`, and inspect live $\text{Beta}(\alpha, \beta)$ posteriors.
- `05 // TASTE & SELF-EVOLVE`: Tune taste parameters, add learned rules, and use the **Self-Improvise Harness Studio** (`POST /api/improvise`), which creates a git snapshot commit (`chore(checkpoint): pre-improvise snapshot...`) before dispatching agents to add new features to `/home/aswath/agy-orchestrator` itself.
- `06 // PHONE AUTOMATION`: Scan OneDrive/Cloud Notes inbox, configure `ntfy.sh` topics, send test phone pushes, and control `agy remote-control`.

Part X: Atomic Component Breakdown, File Map & CLI Reference

10.1 Complete Repository File Map

```
agy-orchestrator/
├── WHITEPAPER.md                    # This comprehensive technical whitepaper
├── README.md                        # Quickstart, architecture overview & CLI cheatsheet
├── package.json                     # Zero-dependency Node v20 ESM manifest
├── bin/
│   ├── orch.mjs                     # Main CLI entrypoint (linked to ~/.local/bin/orch)
│   └── orch.cmd                     # Windows CLI wrapper (linked to %APPDATA%\npm\orch.cmd)
├── config/
│   ├── topology.json                # Live Leader, Router, Goldfish & Subagents configuration
│   └── automation.json              # Phone & Cloud Notes automation channel configuration
├── taste/
│   └── taste-profile.json           # User Taste Contract (ponytail:ultra + brutalist UI rules)
├── memory/
│   ├── orch-mem.db                  # Native ext4 SQLite WAL database (FTS5 + 384-d embeddings)
│   └── Phone-Inbox.md               # Local fallback Markdown inbox for phone automation
├── learning/
│   ├── bandit-state.json            # Thompson Sampling Beta(alpha, beta) posteriors per arm
│   └── records.jsonl                # Execution outcomes & Contrastive DPO preference pairs
├── src/
│   ├── adapters/
│   │   ├── path-bridge.mjs          # WSL <-> Windows path translator & AGY theme sync
```

```
|   |   └── index.mjs                # Pluggable Brain Adapters (agy-wsl, agy-win, cmdc, mcode)
|   ├── topology/
|   |   └── controller.mjs           # Hot-swappable Leader/Subagent manager & EGM auto-pairing
|   ├── memory/
|   |   ├── hybrid_engine.py         # Python SQLite FTS5 BM25 + numpy 384-d RRF + claude-mem bridge
|   |   └── manager.mjs              # Node async interface to hybrid_engine.py & claude-mem worker
|   ├── router/
|   |   └── skill-router.mjs         # 188-skill BM25/trigram router, npx installer & Bandit selector
|   ├── taste/
|   |   └── engine.mjs               # Taste Contract compiler & anti-truncation/anti-slop scorer
|   ├── learning/
|   |   └── loop.mjs                 # Marsaglia-Tsang Beta sampler, reward updater & rule consolidator
|   ├── orchestrator/
|   |   └── engine.mjs               # Full pipeline: Route -> Memory -> Leader -> Workers -> Goldfish
|   ├── automation/
|   |   └── phone-bridge.mjs         # OneDrive Markdown state machine + ntfy.sh + agy remote-control
|   └── ui/
|       ├── server.mjs               # Zero-build HTTP + SSE telemetry & control server (port 4888)
|       └── index.html               # Industrial Brutalist Tactical Telemetry Control Center
├── tools/
|   ├── chronicle.py                 # Aswath Chronicle step-by-step decision & interaction logger
|   └── build_whitepaper_pdf.py      # PyMuPDF (fitz) PDF renderer generating docs/WHITEPAPER.pdf
├── docs/
|   ├── WHITEPAPER.pdf               # Rendered publication-ready PDF whitepaper
|   ├── interactions/
|   |   └── session-ledger.md        # Step-by-step decision & prompt audit trail
|   └── superpowers/
|       ├── specs/2026-11-10-agy-orchestrator-design.md
|       └── plans/2026-11-10-agy-orchestrator-plan.md
└── test/
    ├── topology.test.mjs            # Path translation, theme sync & live leader swap tests
    ├── memory-taste.test.mjs        # FTS5 + dense vector RRF, claude-mem bridge & Taste tests
    ├── router-learning.test.mjs     # 188-skill routing, Thompson Bandit & DPO consolidation tests
    ├── orchestrator.test.mjs        # Adapter CLI flags & full EGM multi-agent pipeline tests
    ├── phone-bridge.test.mjs        # Atomic Markdown inbox state machine idempotency tests
    └── ui-cli.test.mjs              # REST API, SSE stream & CLI command verification tests
```

Part XI: Engineering Chronicle — Decisions, Failures & Pivots

Following the **Aswath Chronicle (`aswath-chronicle`)** protocol, every major architectural decision, bug discovery, and design pivot during the construction of ORCH is recorded below and in [`docs/interactions/session-ledger.md`](docs/interactions/session-ledger.md):

- **Decision 01 — Zero-API-Key Pluggable CLI Brains vs. Paid SDKs**:
- *Context*: User requested a multi-model harness across Gemini and Claude/GPT without paying for API keys.
- *Discovery*: Inspection of the host revealed `agy` (WSL & Windows), `cmdc` v1.88.5, and `mcode` already authenticated locally.
- *Decision*: Built `src/adapters/index.mjs` to wrap headless CLI invocations (`agy -p`, `cmdc -p --yolo --skill`, `mcode exec`).
- **Decision 02 — Pivot from `global-memory.json` to 3-Tier MemGPT + Live `claude-mem.db` Bridge**:
- *Failure Discovered*: Opening `/mnt/c/Users/aswat/.claude-mem/claude-mem.db` from WSL threw `sqlite3.OperationalError: disk I/O error` due to `9p fcntl` lock incompatibility on WAL `-shm` files.
- *Pivot*: Placed write-heavy `orch-mem.db` on native WSL `ext4` while bridging `claude-mem.db` (9,541 observations) via `file:/mnt/c/.../claude-mem.db?immutable=1` and fusing FTS5 BM25 + 384-d `numpy` embeddings via Reciprocal Rank Fusion (RRF).
- **Decision 03 — Sub-10ms Hybrid Skill Router + Thompson Sampling Bandit**:
- *Context*: Calling an LLM subprocess just to choose among 188 local skills adds 10–15s of latency per turn.
- *Decision*: Built a two-stage router combining local BM25 + character trigram cosine similarity with a Marsaglia-Tsang $\text{Beta}(\alpha, \beta)$ Thompson Sampling bandit that enforces dual-category (Gemini + Claude/GPT) participation.
- **Decision 04 — Atomic 3-State Markdown Machine for Phone Automation**:
- *Context*: User wanted phone-to-harness automation via Google Notes / OneDrive / forwarding chat.

- *Decision*: Built `PhoneAutomationBridge` watching `/mnt/c/Users/aswat/OneDrive/Orch-Phone-Inbox.md` (with atomic `- [ ]` → `- [~]` → `- [x]` transitions) plus `ntfy.sh` pub/sub streaming and `agy remote-control`.
- **Decision 05 — Zero-Build Tactical Telemetry Web UI (`industrial-brutalist-ui`)**:
- *Context*: User needed a rich visual dashboard to switch leaders, inspect tasks, and self-evolve the harness without heavy bundler bloat (`/ponytail [ultra]`).
- *Decision*: Engineered `src/ui/server.mjs` and `src/ui/index.html` using pure `node:http` + Server-Sent Events (SSE) in `#0A0A0A` CRT carbon / `#E61919` aviation red telemetry styling with git-checkpointed self-evolution.